

Today's Agenda :-

will start at 7:05AM

- 1) Find k th smallest element in BST
- 2) Morris Inorder Traversal
- 3) Find an element in a Binary tree
- 4) Path from root to node
- 5) Lowest Common Ancestor (LCA) of 2 Nodes
- 6) Lowest Common Ancestor (LCA) in BST
- 7) In-time & Out-time in a Binary tree
- 8) LCA(x, y) for multiple queries.

Difference between Binary Tree & Binary Search Tree :-

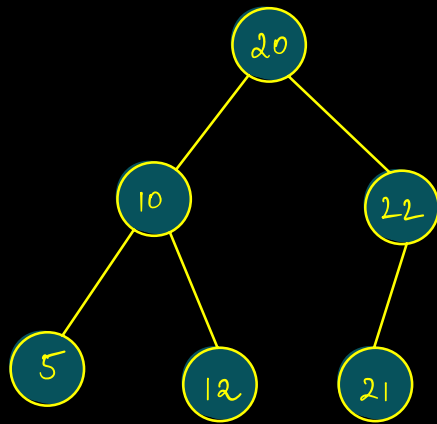
Binary Tree :- Hierarchical data structure where each node can have at max 2 children.

Binary Search Tree :- Type of binary tree with an additional property :-

for all nodes,

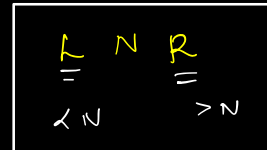
$$\begin{array}{c} \text{All ele} \\ \text{in} \\ \text{LST} \end{array} \leq \text{Node} \leq \begin{array}{c} \text{All ele} \\ \text{in} \\ \text{RST} \end{array}$$

Q1) Find Kth smallest ele in a Binary Search tree



K = 1 : 5

K = 3 : 12



→ 5 10 12 20 21 22

Appl: Do inorder & find Kth smallest

TC: O(N)

SC: O(N)

int cnt = 0, ans = -1

void inorder(Node root, int k)

if (root == NULL) return;

inorder(root->left, k)

// access ele / Node

cnt++

if (cnt == k) { ans = root->data; return; }

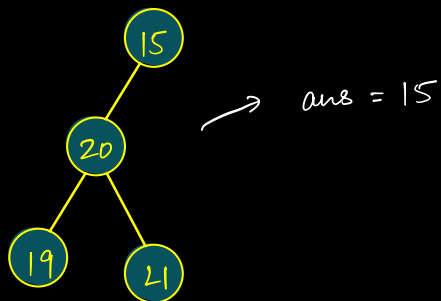
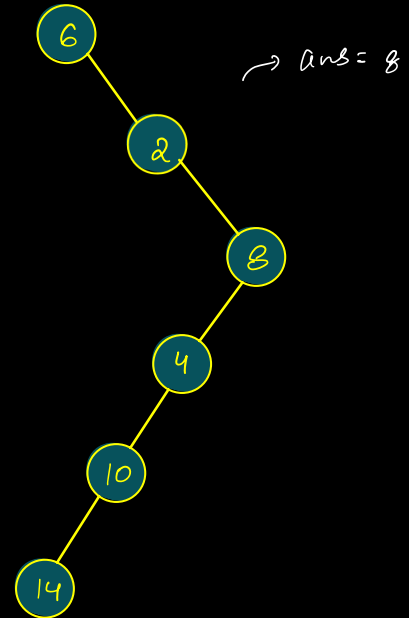
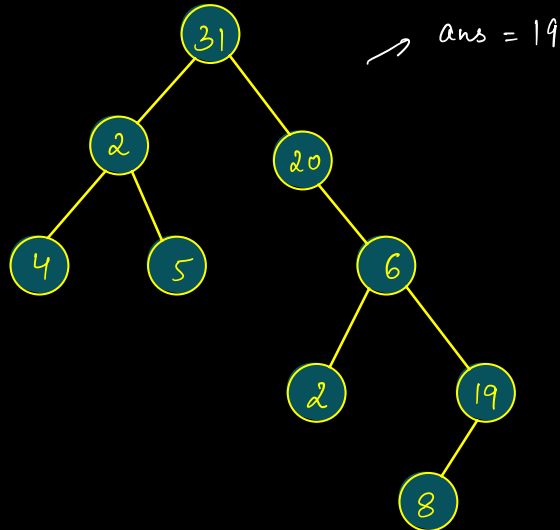
inorder(root->right, k)

}

Recursive
inorder.

Q2) With inorder traversal on a tree, what's the last node we print?

L N R



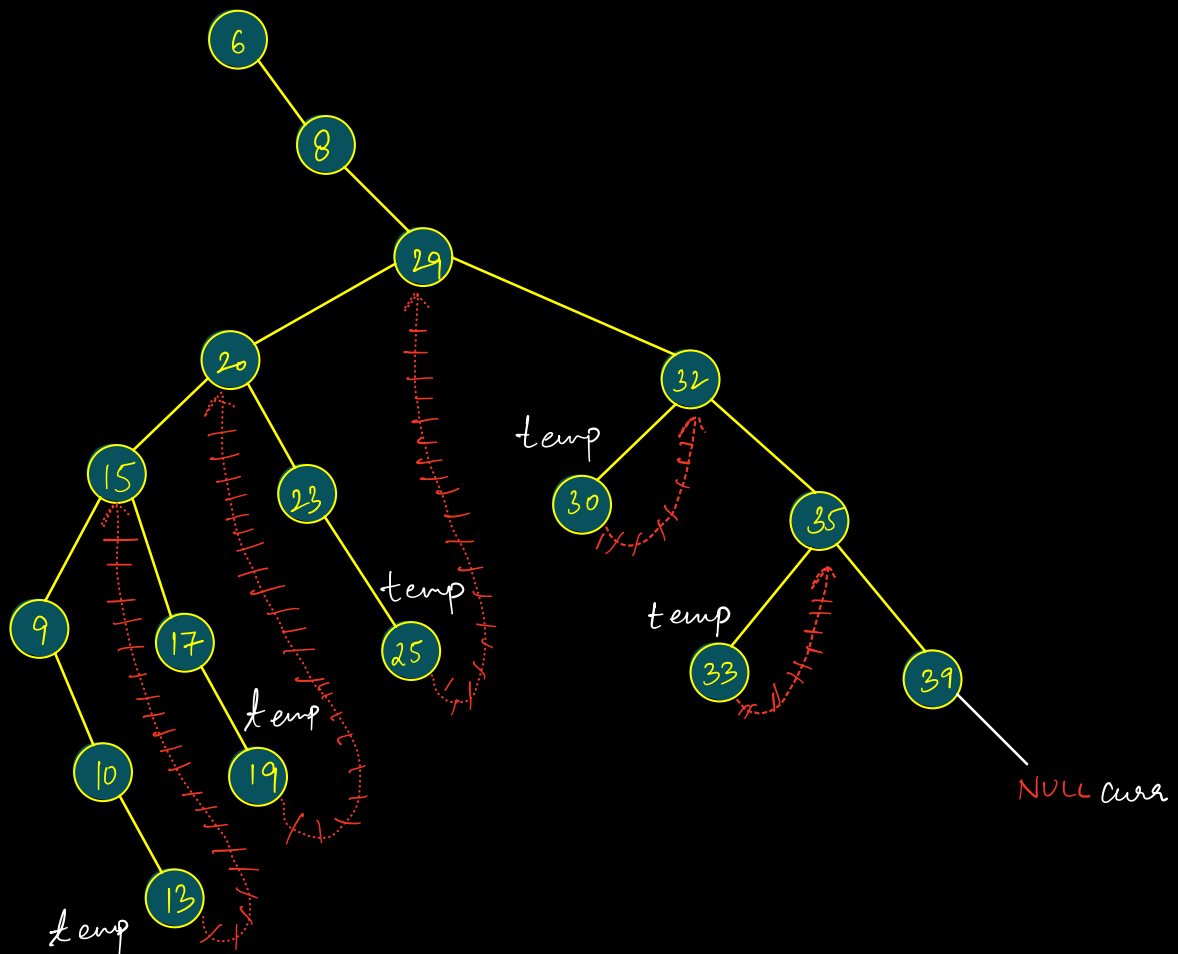
Inorder.

	TC	SC
<u>Recursive</u>	$O(N)$	$O(H)$ ✓
<u>Iterative</u>	$O(N)$	$O(H)$

for all B.Ts.

Q3) Morris Inorder Traversal :-

6 8 9 10 13 15 17 19 20
23 25 29 30 32 33 35 39



temp right = curr

Pseudo Code :-

```
while (curr != NULL) {
```

```
    if (curr.left == NULL) {
```

```
        print (curr.data)
```

```
        curr = curr.right
```

```
    }
```

```
    else {
```

```
        Node temp = curr.left
```

```
        while (temp.right != NULL && temp.right != curr) {
```

```
            temp = temp.right
```

```
        }
```

```
        if (temp.right == NULL) { // coming to curr for first time
```

```
            temp.right = curr
```

```
            curr = curr.left
```

```
        }
```

```
    } // coming to 2nd time.
```

```
        temp.right = NULL
```

```
        print (curr.data)
```

```
        curr = curr.right
```

```
    }
```

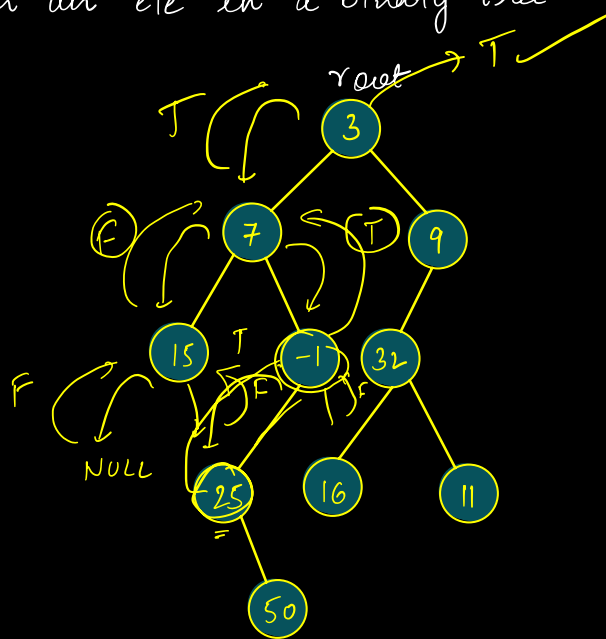
```
}
```

```
}
```

TC : $O(N)$

SC : $O(1)$

Q3) Search an ele in a Binary tree



Search(25) : true

```

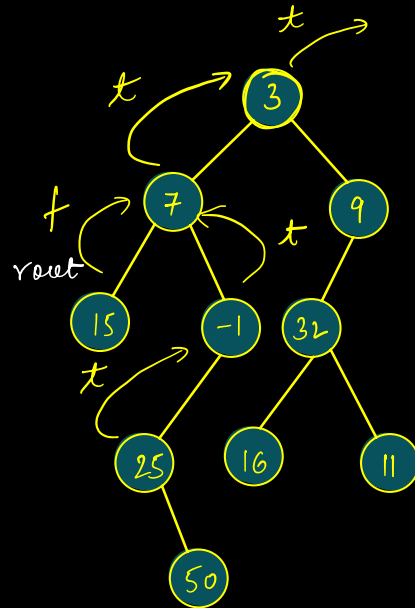
bool search ( Node root, int k) {
    if (root == NULL) { return false }

    if (root->data == k) {
        return true
    }

    if (search(root->left, k) || search(root->right, k))
        return true
    else {
        return false
    }
}
    
```

Tc: O(N)
Sc: O(h)

Q4) Path from root to Node (All ele are distinct)



target = 25 ✓
[3 7 -1 25]

↓
[25 -1 7 3] ✓
reverse &
get ans.

list < int > ls

```
bool search ( Node root, int k) {
    TC: O(N)
    if (root == NULL) { return false }
    if (root->data == k) {
        ls.push(root->data); return true
    }
    if (search(root->left, k) || search(root->right, k))
        ls.push(root->data); return true
    else {
        return false
    }
}
```

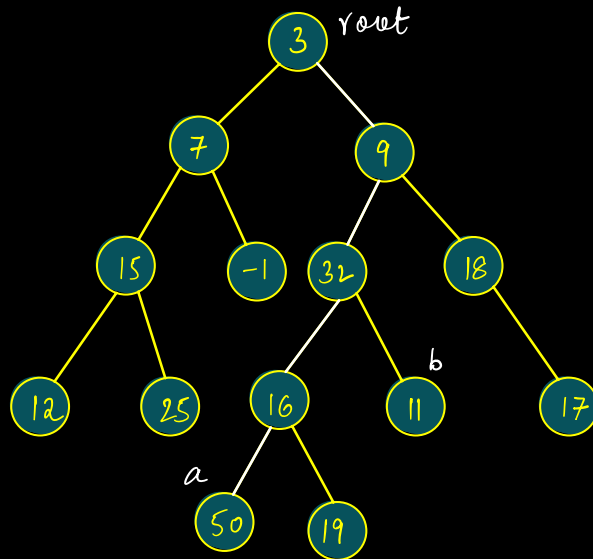
Reverse the list to get path from Root till given node

Q5) Find LCA (a,b) in a Binary tree Note: A Node is also an ancestor of itself

Lowest Common Ancestor

Least

Common



$$LCA(16, 18) = 9$$

$$LCA(50, 19) = 16$$

$$LCA(11, 9) = 9$$

$$LCA(25, 9) = 3$$

$$\underline{LCA(50, 11)} = 32$$

$LCA(a, b)$

LCA



Path from root till a : $\rightarrow [3, 9, 32, 16, 50]$

Path from root till b : $- [3, 9, 32, 11]$

App:

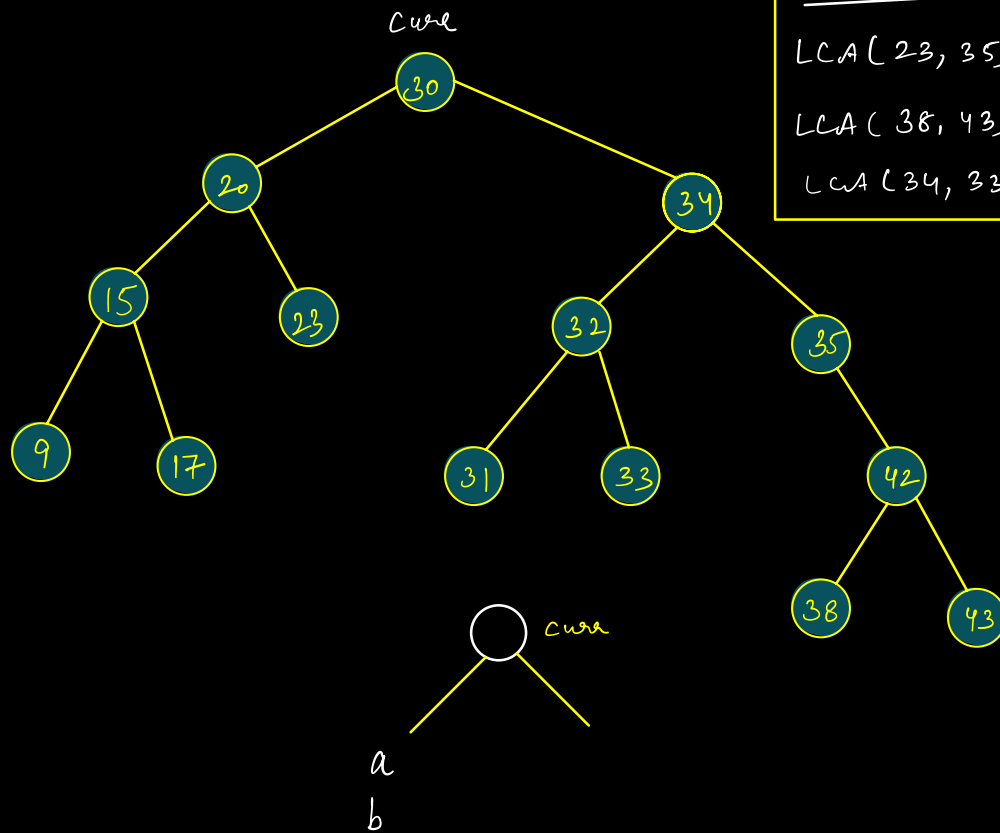
Paths from root till a & root till b

Traverse the path & find the last common node

TC: $O(N)$

SC: $O(N)$

Q6) LCA in a BST :-



$$\text{LCA}(31, 43) = 34$$

$$\text{LCA}(23, 35) = 30$$

$$\text{LCA}(38, 43) = 42$$

$$\text{LCA}(34, 33) = 34$$

$a, b = \text{valid Nodes}$

$$T_C : \mathcal{O}(H)$$

SC: OC12

Node cur = root

```

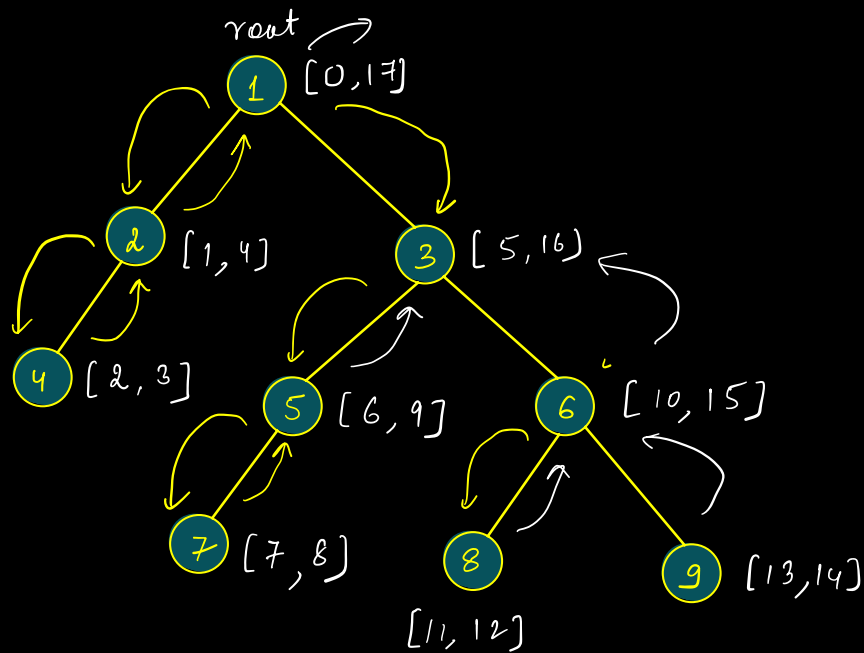
while (true) {
    if (curr.data > a.data && curr.data > b.data) {
        curr = curr.left
    }
    else if (curr.data < a.data && curr.data < b.data) {
        curr = curr.right
    }
    else {
        break
    }
}

return curr

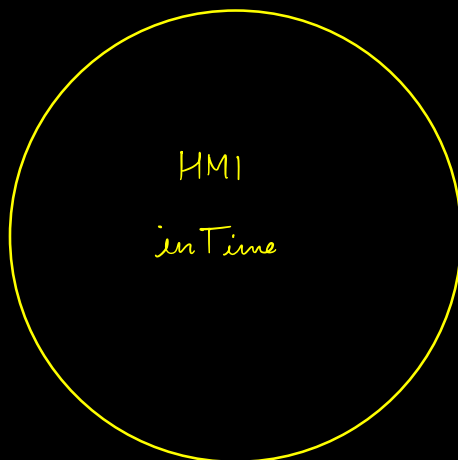
```

Q.7) In-time & Out-time in a Binary tree Concept :-

$t = 18$



for all the Nodes, store in-time & out-time.



HashMap < Node, int > hm1 // for inTime

HashMap < Node, int > hm2 // for outTime

int t = 0

void traverse(Node root) {

TC: $O(N)$

if (root == NULL) return;

hm1.insert({root, t}) // record inTime

t++

traverse(root->left)

traverse(root->right)

hm2.insert({root, t}) // record outTime

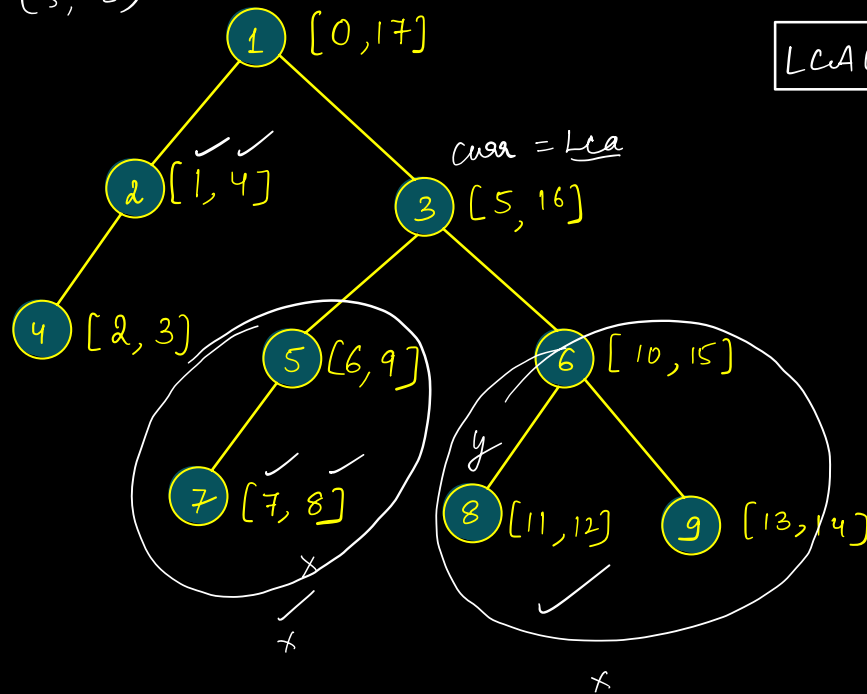
t++

}

LCA(a, b) $\rightarrow O(N)$
 $O(1)$

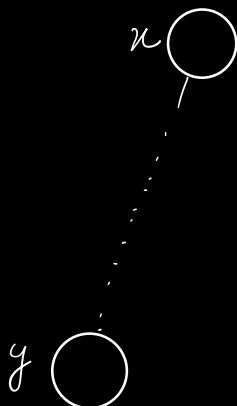
Q 8) LCA(a,b) for multiple queries in a Binary tree

$\begin{matrix} a & b \\ (5, & 8) \end{matrix}$



x, y

whether x is an ancestor of y using in & out time of x & y .



$$\begin{matrix} in(x) < in(y) \\ \&\& \\ out(x) > out(y) \end{matrix}$$

bool isAncestor(x, y)

```

|   return hml[x] <= hml[y] && hml[x] >=
|   hml[y];
|
|   }

```

if queries(a, b)

curr = root

while (true) {

if (isAncestor(curr.left, a) && isAncestor(curr.left, b)) {

curr = curr.left

}

else if (isAncestor(curr.right, a) && isAncestor(curr.right, b)) {

curr = curr.right

}

else {

return curr

}

}

→ TC for 1
query : $O(H)$

$Q * H$ better

than

$Q * N$??

TC: $N + Q * H$

SC: $O(N)$

Observations

- **In-Order Traversal:**
It visits Binary Search Tree (BST) nodes in ascending order, enabling efficient kth smallest element retrieval.
- **Morris Traversal:**
An efficient memory-saving tree traversal method with $O(1)$ space complexity.
- **Path from Root:**
DFS traversal is used to find the path from the root to a node, with space complexity tied to recursion depth.
- **Lowest Common Ancestor (LCA) in Tree:**
LCA is found through recursion with $O(n)$ time complexity and stack space.
- **In-Time & Out-Time:**
These values in DFS help create hierarchical visualizations of trees.
- **Interval Assignment Visualization:**
Provides a visual hierarchy for analyzing complex structures in various fields.
- **Finding LCA for Multiple Queries:**
LCA retrieval for multiple pairs involves adjusting node depths until they meet.

Thank You 😊

